Module	Fundamentals of Machine Learning: EEEM066
Assessment type	Coursework assignment
Weighting	20%
Deadline	4:00PM, Friday, 6 December 2024 (Week 11)
STUDENT URN	6528234
STUDENT Name	Md Abrar Fahim

Objectives:

- Learn and become familiar with the process of sequential hyperparameter tuning of an entire deep learning pipeline.
- Develop intuition via experimentation on how to use such model design and parameter tuning to improve the performance of a pipeline.
- Interpret experimental results of deep learning experiments and their implications.

Outcomes:

- Navigate general deep learning codebases and modify according to requirements.
- Empirically suggest reasonable values for hyperparameters and select the ones enabling the best performance.
- Understand the implications of changing the backbone architecture of a pipeline.
- Reason why data augmentation works.
- Understand the implications and importance of selecting the best learning rate and batch size.
- Understand how to report experimental results professionally. This includes using tables, plots, and diagrams to communicate ideas and results.

Knife classification in real-world images

For this coursework assignment, you are encouraged to apply what you have learnt from the module materials and your participation in collaborative learning activities like class discussions.

Guidelines

- Submit your assignment as a **Word or PDF document**, along with the supporting evidence as required, via the SurreyLearn submission page before the deadline. Do not submit any other unrelated documents that could flood your submission and mislead the marking process. Only the latest version submitted before the deadline will be assessed if multiple versions exist.

- Label your submission file as: EEEM066-[insert your URN number]-Assignment e.g., EEEM066-1234567-Assignment

- Include a reference list at the end of your assignment, and use the Harvard or IEEE referencing style when citing sources. Please use the selected option consistently.

- **Word limit:** Each section of the report should not exceed **200 words** (excluding tables, plots, and graphs). Penalties of up to 50% may be applied for unnecessarily long reports.

- This assignment requires time and effort. It is recommended that you start working on it early to minimise the possibility of experiencing stress around it and avoid missing the deadline.

- You will receive both an overall mark and written feedback on your assignment.

- Extensions for this assignment will only be granted under exceptional circumstances. Extension requests must be submitted via the Extenuating Circumstances (ECs) process, as tutors are not permitted to approve ad hoc extension requests.

Note:

Please follow Surrey's guidelines for avoiding plagiarism.

You must complete this coursework individually. Copying code or text from the internet or another student and including it in your assignment without proper attribution constitutes plagiarism.

Undetected plagiarism undermines the quality of your degree by hindering the assessor's ability to evaluate your work fairly and prevents you from learning through the coursework. If plagiarism is suspected, you will be referred to an Academic Misconduct Panel, which may result in academic sanctions.

All assignments will be checked for plagiarism using Turnitin. Therefore, it is important that you correctly reference sources that have informed your work where appropriate.

Task

Review the instructions for the assignment carefully before starting. This assignment is marked out of 100 and will contribute **20%** to your total module grade. The three sections of the assignment are worth **90 marks**, with an additional **10 marks** allocated to the presentation and clarity of the report.

Background

In recent years, the number of crimes involving sharp objects, such as knives, has increased all across the world. In the year ending March 2022, England and Wales saw around 45,000 offences involving a knife or sharp instrument. This was 9% higher than in 2020/21 and 34% higher than in 2010/11.

Therefore, there is a need to develop an AI-based weapon analysis system that enables officers to identify the make, model, and seller of knives using photographs. Although functional, current knife recognition and labelling methods are beset by inefficiencies due to strained public resources, the possibility of human error, and the UK's expanding digital footprint.

The following figure provides a visual representation of a knife classification system.

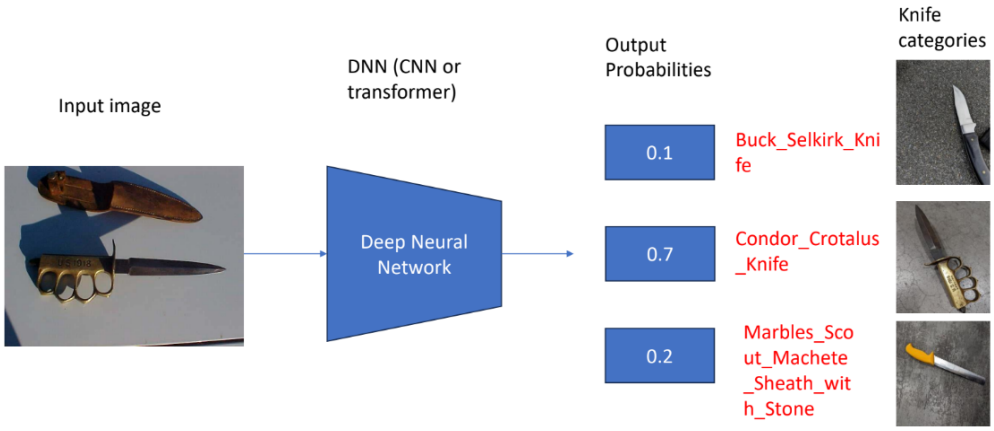

Figure 1: Overview of a knife classification system.

Instructions

In this assignment, you will develop a deep-learning-based system to classify knife images into different categories i.e., classes. You will gain real-world experience in designing, training, and evaluating a deep neural network for accurate, fine-grained knife classification.

You are required to document your findings in a report, divided into three sections as outlined later. Complete each section (in this document) according to the provided guidance.

Dataset

Use the "Knife classification" training dataset (provided here), which includes approximately 9,928 knife images categorised into 192 classes, along with a test dataset consisting of 351 images to complete the assignment. The performance of your model will be evaluated using the mean Average Precision (mAP) metric.

Important note:

Please do not distribute this dataset or use it for any purposes other than this assignment.

Compute platform

To complete this coursework assignment, you will need to use Google Colab. You are welcome to revisit the lab sheets you engaged with during this module for further guidance that can support you in completing this assignment.

Getting started

To prepare for this assignment, it is recommended that you engage with the following readings, available via the Reading List, before starting work on developing your knife classification system:

- "Fine-grained image analysis with deep learning: A survey" by Wei Xiu-Shen, Jianxin Wu, and Quan Cui
- "Weapon classification using deep convolutional neural network" by Neelam Dwivedi, Dushyant Kumar Singh, and Dharmender Singh Kushwaha

Additionally, you are encouraged to revisit the lectures in this module and the lab sessions on using Python and PyTorch to train and test CNNs for image classification. The PyTorch Introductory lab sheet and the lecture slides on these topics are available on SurreyLearn.

Support in programming

You can consult this GitHub repository for a reference code base. Should you experience any problems during the development of your knife classification, please feel free to open an issue.

Deliverables

For this assignment, you are required to run your project in the Colab. For the report, you must document the experiments performed using your software.

Additional instructions

Keep the following guidance in mind as you work on producing the deliverables for this assignment:

Hyperparameters

For the parameters that are not learnable, you can set them before starting the training.

Format

- Ensure your responses to each question are on the spot as shown in Figure 2. Focus on the specific question and not move away.

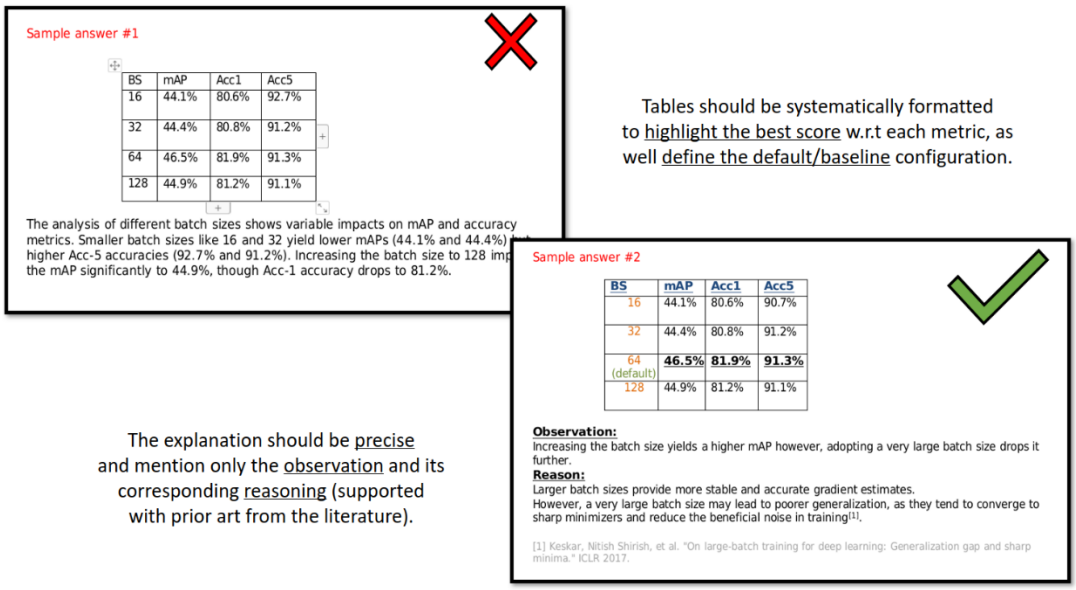

Figure 2: Response example.

Log files

- Submit your own log files from your codebase. You are required to submit a separate log file for each experiment-based question.
- Do not change the log file structure. Each log file is watermarked and unique to each run.
- Name each log file according to the section number and question number: (log_sec{Section_num}_q{Question_num}.txt). For example, the log file generated for an experiment that corresponds to Section 1 Question 2, should be named as “log_sec1_q2.txt”.

Presentation and clarity

In addition to the technical content, the presentation and clarity of your writing will be assessed.

Model performance metrics

While different metrics are available, it is recommended that you prioritise mAP when selecting the best-performance models.

Start your assignment

Write your assignment in the designated space provided below. Each of the sections outlines the tasks required and specifies the information you must include in your report for that section.

Section 1: Familiarity with the code provided (40 marks)

1. Run the code using the default settings. Discuss the steps involved in the training and evaluation process, and explain the impact of the observed performance using an appropriate metric. (20 marks)
2. Apply a different CNN variant from the one used in the code provided. Critically compare the results with those observed in Question 1. (10 marks)
3. Apply one more neural network architecture different from those in Questions 1 and 2. Critically discuss and compare the results from these previous two steps. (10 marks)

Note:

For Questions 2 and 3, ensure the new architecture is explicitly specified in your shell script command if you are using one from the codebase.

Start writing here:

The data loader loads the pre-processed dataset from the knifeDataset class. The –model_mode command initialises the model. The training goes through 20 epochs to optimise the weights. Evaluation computes predictions and calculates measurable metrics.

Mean average precision score (mAP) is used for evaluation. It assesses both the precision and recall of the model's performance, ensuring a balanced approach. It is scaled from 0 (poor performance) to 1 (perfect performance) [1].

Mode	Train Loss	Valid mAP	Time
Train	0.022	0.516	55 min 29 sec
Validation	0.022	0.561	56 min 07 sec

The mAP score shows that the model performs moderately well but makes lots of incorrect predictions. Limited transformations may cause underfitting or overfitting, the architecture itself may be limited at capturing complex features. Can be resolved by experimenting with different architectures and adding transformations to the data.

Model Architecture	Validation mAP Score	Time
effecientnet_b0	0.561	56 min 07 sec

resnet50	**0.603**	**67 min 54 sec**

Resnet50 has the better performance. It has a deeper architecture; therefore, it can capture more of the complex and nuanced features. However, it's more computationally expensive where training time is longer [2].

Model Architecture	Validation mAP Score	Time
effecientnet_b0	0.561	56 min 07 sec
resnet50	**0.603**	**67 min 54 sec**
vgg16	0.063	70 min 47 sec

Vgg16 performs the worst, where the model is randomly guessing. It cannot detect small feature maps like slender edges and is the most computationally expensive architecture, as it has more parameters [3]. Effecientnet_b0 is the most efficient but provides mediocre results. Resnet50 has the best performance but is computationally expensive.

Section 2: Dataset preparation and augmentation experiment (30 marks)

Begin with the default data augmentation setting of the default command line, which uses colour jittering.

1. Further append on top two additional data augmentation techniques, one at a time e.g., Default + “random rotation”, Default + “horizontal flip”. Compare the results with the default configuration in the provided code and discuss the differences in performance. (20 marks)

2. Combine the augmentation techniques from Question 1 to find the best-performing combination. Highlight any improvement or drop in the overall score. (10 marks)

Start writing here:

Transformation	Validation mAP Score
Default	**0.603**
Default + “random rotation 15”	0.600
Default + “vertical flip 0.5”	0.521

Random rotation is applied where images are rotated randomly by 15° so the model can generalise better to orientation changes. The slight dip in performance could be due to the model not being able to capture consistent features due to the random rotations. There may also be some slight noise added due to some of the rotations not looking natural to the model. However, it is only a small dip, meaning there is still some variation, allowing the model to still generalise well [4].

Vertical flip flips the image in its vertical axis, where each image has a 50% probability of being flipped, so the model can generalise better to flipped images. The worse performance can be due to the flipped features preventing the model from associating it with its correct class. The model may also be confused about which orientation is correct, as it processes both original and flipped images [5].

Transformation	Validation mAP Score
Default	0.603
Default + “random rotation 15”	0.600
Default + “vertical flip 0.5”	0.521
Default + “random rotation 15” + “vertical flip 0.5”	**0.640**

The combination of all three augmentations provides the best performance (0.640), as they complement each other better, allowing the model to generalise better on unseen data as it learns orientation-invert along with different lighting features. This makes the data more diverse, and applicable for real-world scenarios [6].

Section 3: Exploration of Hyperparameters (20 marks)

Start with the default learning rate (LR) and batch size (BS).

1. Exploration of LR. (10 marks)
    a. Experiment with three values of LR (in addition to the default value).
    b. Discuss the observed impact of each value on overall performance.

2. Exploration BS. (10 marks)
    a. Using the best LR value from the experiments in Question 1, test three different values of the BS in addition to the default value.
    b. Discuss how the changes in BS affect overall performance.

Start writing here:

Learning Rate	mAP Score
0.00005 (Default)	0.640
0.000001	0.077
0.00001	**0.685**
0.0001	0.517

Decreasing the learning rate to 0.00001, produces the best performance as the weights are updated more slowly but more precisely, where skipping over the optimal solution is avoided, and feature representations in the final stage are fine-tuned more precisely, thus allowing the model to better generalise. Decreasing the learning rate too much produces a poor performance, as the weight updates are too slow, causing the model to be stuck in a suboptimal region and remain far from the optimal region, causing underfitting. Increasing the learning rate too much also produces poor performance as the larger weight updates may cause the model to go over the optimal region, causing overfitting [7].

Batch Size	mAP Score	Time
32 (Default)	0.685	56 min 39 sec
64	0.649	58 min 17 sec
16	**0.704**	**59 min 15 sec**
8	0.708	66 min 09 sec

Batch size 8 provides the best performance, as the decrease in size allows for beneficial noise to be introduced during gradient updates, increasing diversity, thus improving generalisation. However, it is more computationally expensive as there are more iterations of weight updates per epoch. Increasing the batch size allow for more stable gradients, but there is less noise, therefore worse generalisation. However, there are fewer iterations for weight updates, making it more efficient. Batch size 16 is the optimal choice as it has similar performance to 8 but more efficient [8].

References:

[1] D. Shah, “Mean Average Precision (mAP) Explained: Everything You Need to Know,” *www.v7labs.com*, Mar. 07, 2022. https://www.v7labs.com/blog/mean-average-precision (accessed Dec. 01, 2024).

[2] “ResNet: Residual Neural Networks - easily explained! | Data Basecamp,” Mar. 18, 2023. https://databasecamp.de/en/ml/resnet-en (accessed Dec. 01, 2024).

[3] “VGG16: A Brief Summary | Kaggle,” *Kaggle.com*, 2024. https://www.kaggle.com/discussions/getting-started/178568 (accessed Dec. 01, 2024).

[4] Jacob Solawetz “Why and How to Implement Random Rotate Data Augmentation,” *Roboflow Blog*, Jun. 24, 2020. https://blog.roboflow.com/why-and-how-to-implement-random-rotate-data-augmentation/ (accessed Dec. 01, 2024).

[5] “Vertical Flip | CloudFactory Computer Vision Wiki,” *CloudFactory Computer Vision Wiki*, 2024. https://wiki.cloudfactory.com/docs/mp-wiki/augmentations/vertical-flip (accessed Dec. 01, 2024).

[6] “What is Data Augmentation? - Data Augmentation Techniques Explained - AWS,” *Amazon Web Services, Inc.* https://aws.amazon.com/what-is/data-augmentation/ (accessed Dec. 02, 2024).

[7] J. Brownlee, “Understand the Impact of Learning Rate on Neural Network Performance,” *Machine Learning Mastery*, Jan. 24, 2019. https://machinelearningmastery.com/understand-the-dynamics-of-learning-rate-on-deep-learning-neural-networks/ (accessed Dec. 02, 2024).

[8] J. Brownlee “How to Control the Stability of Training Neural Networks With the Batch Size,” *Machine Learning Mastery*, Aug. 28, 2020. https://machinelearningmastery.com/how-to-control-the-speed-and-stability-of-training-neural-networks-with-gradient-descent-batch-size/ (accessed Dec. 02, 2024).

Rubric

Section 1: Familiarity with the code provided (40 marks)

Task 1.1: Running and understanding the code (20 marks)

- **Understanding the code execution**
  - *Criteria*: Clear understanding and explanation of the code flow and processes.
  - *Indicators*: Detailed description of data loading, model initialisation, training loop, and evaluation steps.

(5 marks)

- **Metric explanation and performance analysis**
  - *Criteria*: Appropriate choice and understanding of the evaluation metric(s) (e.g., accuracy, mAP).
  - *Indicators*: Correct calculation and interpretation of the metric values. Discussion on what these values imply about the model's performance.

(10 marks)

- **Discussion of observed performance**
  - *Criteria*: Insightful discussion on the observed results.
  - *Indicators*: Analysis of performance patterns, potential overfitting/underfitting issues, and suggestions for improvement.

(5 marks)

Task 1.2: Applying a different CNN variant (10 marks)

- **Implementation of a new CNN variant**
  - *Criteria*: Correctly modifying the code to implement a different CNN architecture.
  - *Indicators*: Model architecture is different from the provided one, and code runs without errors.

(4 marks)

- **Comparison and critical analysis**
  - *Criteria*: Critical comparison between the original and new CNN results.
  - *Indicators*: Use of quantitative results to support comparison, discussion on potential reasons for performance differences.

(6 marks)

Task 1.3: Applying another neural network architecture (10 marks)

- **Implementation of a new neural network**
  - *Criteria*: Correctly implementing a neural network architecture different from those in Tasks 1.1 and 1.2.
  - *Indicators*: Model architecture is distinct, and code runs correctly.

(4 marks)

- **Comprehensive comparison and analysis**
  - *Criteria*: Deep comparative analysis between all three architectures.
  - *Indicators*: Insightful discussion on the strengths and weaknesses of each model, based on quantitative results and qualitative observations.

(6 marks)

Section 2: Dataset preparation and augmentation experiment (30 marks)

Task 2.1: Augmentation techniques (20 marks)

- **Implementation of additional augmentations**
  - *Criteria*: Successful addition of two new augmentation techniques.
  - *Indicators*: Correct application and explanation of chosen techniques, code runs without errors.

(8 marks)

- **Performance comparison and analysis**
  - *Criteria*: Comparative analysis of the impact of each augmentation setting.
  - *Indicators*: Clear explanation of performance differences between default and new settings, supported by quantitative results.

(8 marks)

- **Discussion on results and insights**
  - *Criteria*: Insightful interpretation of how augmentations affected model performance.
  - *Indicators*: Discussion on potential reasons for performance changes and implications for model robustness.

(4 marks)

Task 2.2: Best-performing augmentation combination (10 marks)

- **Combining techniques and experimentation**
  - *Criteria*: Correct implementation of combined augmentation techniques.
  - *Indicators*: Code successfully combines the techniques and executes without errors.

(5 marks)

- **Analysis of improvement or decline**
  - *Criteria*: Detailed discussion on whether the combined augmentation improved performance.
  - *Indicators*: Use of appropriate metrics to highlight any performance change, supported by logical reasoning.

(5 marks)

Section 3: Exploration of hyperparameters (20 marks)

Task 3.1: Learning rate exploration (10 marks)

- **Experimentation with different LR values**
  - *Criteria*: Correct experimentation with three additional LR values.
  - *Indicators*: Clear documentation of the chosen LR values and implementation.

(4 marks)

- **Performance analysis of each LR**
  - *Criteria*: Thorough analysis of the effect of each LR value on performance.
  - *Indicators*: Use of metrics to compare performance across LR values, discussion on the reasons behind observed changes.

(6 marks)

Task 3.2: Batch size exploration (10 marks)

- **Experimentation with different BS values**
  - *Criteria*: Correct experimentation with three additional BS values.
  - *Indicators*: Clear documentation of the chosen BS values and implementation using the best-performing LR from Task 3.1.

(4 marks)

- **Performance analysis of each BS**

- *Criteria*: Detailed analysis of the impact of each BS value on performance.
- *Indicators*: Use of metrics to compare performance across BS values, discussion on the reasons behind observed changes.

(6 marks)

Report writing and presentation (10 marks)

- **Figures should be well-presented, with readable axes and labels.**

(2 marks)

- **The writing should be clear, grammatically correct, and easy to follow.**

(6 marks)

- **Tables are used when discussing results involving multiple hyperparameters values**
  - *Criteria*: Clarity, structure, and depth of the written report.
  - *Indicators*: Logical flow, clear explanation of methods and results, proper use of figures/tables to support the analysis.

(2 marks)

Total: 100 marks